

Skill 7 — Program 1: waitpid() Process Synchronization

Problem

Create a child process, allow it to execute a command, and make the parent monitor it using `waitpid()`.

```

    return 0;
^C
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Skill7.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill7.c -o skill7
cc1: fatal error: skill7.c: No such file or directory
compilation terminated.
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Skill7.c -o Skill7
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Skill7
Parent process started.
Parent waiting for child...
Child process started.
Child process is executing.
Hello from child process!
Child exited normally.
Exit status: 0
Parent received child status.
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill7.c
cat: skill7.c: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Skill7.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {

    pid_t pid;
    int status;

    printf("Parent process started.\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {

```



saikrishna__90156@DESKTOP- X



```
if (pid < 0) {
    perror("fork failed");
    return 1;
}

if (pid == 0) {

    printf("Child process started.\n");

    sleep(2);

    printf("Child process is executing.\n");

    execlp("echo", "echo",
           "Hello from child process!", NULL);

    perror("exec failed");
    exit(1);
}

else {

    printf("Parent waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Child exited normally.\n");
        printf("Exit status: %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent received child status.\n");
}

return 0;
}
```

saikrishna__90156@DESKTOP-2JF9H5T:~\$ |